

Contents

Deliverable 1 — Formal Embedding Design	1
1. Notation and grounding	1
2. The embedding functions	2
3. State-capability relationships	3
4. Compatibility (directed) — precondition-effect <i>and</i> input-output	3
5. Similarity (symmetric) — and why it is not composability	4
6. Composition	4
7. Goal relevance	5
8. Operational properties — embedded or separate?	5
9. Complexity	6
10. How the eight required properties are met	6

Deliverable 1 — Formal Embedding Design

Representation name: *Typed-Atom Partitioned Capability Embedding (TAPCE)*

One-sentence idea: ground every typed state variable into a small set of *atoms* (Boolean propositions), represent states, goals, preconditions, constraints and effects as **ternary vectors** over those atoms, represent data dependencies as **port vectors**, keep operational properties in a **monoid-structured block**, and define *composition* block by block so that it is an associative operation with an identity element that is **sound with respect to real execution**.

1. Notation and grounding

An application is $A = (S, C, S_I, G, R, K)$ with capabilities $C_i = (T_i, I_i, O_i, P_i, E_i, K_i, R_i, Q_i, \text{Rel}_i, A_i, M_i)$ exactly as in the assignment.

1.1 Atom universe

Every state variable x is grounded into atoms; the union is the atom universe $\mathcal{A}$, $|\mathcal{A}| = d_a$.

variable type	atoms	meaning of an atom
Boolean x	one atom a_x	" x is true"
enumerated $x \in \{v_1 \dots v_m\}$	m atoms $a_{x=v_j}$	" $x = v_j$ " (mutually exclusive)
integer n	one atom $a_{n \geq c}$ for each cut-point c occurring in the specification	" $n \geq c$ " (monotone: $n \geq c_2 \Rightarrow n \geq c_1$ for $c_1 < c_2$)

Cut-points are discovered automatically from all comparisons in preconditions, constraints and goals ($>$, $>=$, $<$, $<=$, $=$) are normalised to atoms $n \geq c$; reals must be discretised to integer units).

A **ternary vector** $u \in \mathbb{T} = \{-1, 0, +1\}^{d_a}$ states: $+1$ the atom must/does hold, -1 it must/does not hold, 0 unconstrained / unchanged / unknown.

Two operators on ternary vectors (entry-wise):

$$(u \sqcup w)_k = \begin{cases} u_k & u_k \neq 0 \\ w_k & \text{otherwise} \end{cases} \quad (u \oplus w)_k = \begin{cases} w_k & w_k \neq 0 \\ u_k & \text{otherwise} \end{cases}$$

$\sqcup$ merges two *requirement* vectors (contradiction is detectable: $u_k w_k = -1$); $u \oplus w$ is *override*: apply effect w to state/knowledge u .

The **saturation operator** $\sigma : \mathbb{T} \rightarrow \mathbb{T}$ adds literals implied by typing: for an enum group, a $+1$ forces all siblings to -1 (and if all but one are -1 the last becomes $+1$); for integer atoms, $+1$ at c propagates to all lower cut-points and -1 at c to all higher ones.

Grounding functions: $\ell(\text{literal}) \in \mathbb{T}$ gives the *primary* atoms of a condition (e.g. `Cart.item_count > 0` $\mapsto +1$ at $a_{\text{count} \geq 1}$; `User.role` in `{CUSTOMER, ADMIN}` $\mapsto -1$ at $a_{\text{role}=\text{GUEST}}$); $\alpha(x := v) \in \mathbb{T}$ gives the *complete* assignment of the variable's group (e.g. `Payment.status := SUCCESS` $\mapsto +1$ at `SUCCESS` and -1 at all other statuses).

1.2 Ports

Inputs/outputs (name, type, domain, required) are keyed $\kappa = \text{name:type:domain}$. With a domain hierarchy (child < parent, e.g. `positive_decimal` < `decimal`), the port vocabulary Π has $d_p = |\Pi|$ entries. An **output** is stored *closed upward*: producing `amount:DECIMAL:positive_decimal` also sets `amount:DECIMAL:decimal`. So “output satisfies input” is an exact membership test (inner product) even with domain subsumption.

2. The embedding functions

2.1 States — $\phi_S : S \rightarrow \{-1, +1\}^{d_a}$

$$\phi_S(S) = \sigma\left(\bigoplus_{(x,v) \in S} \alpha(x := v)\right)$$

A complete state is a bipolar vector; a partial state has zeros for unknown variables. Exactness: ϕ_S is injective on states up to the discretisation induced by the cut-points.

2.2 Goals — $\phi_G : G \rightarrow \mathbb{T}$

$$\phi_G(G) = \bigsqcup_{g_j \in G} \ell(g_j)$$

Satisfaction (progress): $\text{prog}(s, \gamma) = \frac{|\{k : \gamma_k \neq 0, s_k = \gamma_k\}|}{|\{k : \gamma_k \neq 0\}|}$; $S \models G \iff \text{prog} = 1$.

2.3 Capabilities — $\phi_C : C \rightarrow \mathbb{R}^d$ (partitioned)

$$\phi_C(C_i) = (\underbrace{p, \kappa, e, \mathbf{i}, \mathbf{o}}_{\text{functional blocks}}, \underbrace{\tau, \mu, \rho}_{\text{implementation blocks}}, \underbrace{q^+, q^\circ}_{\text{operational blocks}})$$

block	space	definition
p	$\mathbb{T}$	preconditions $\bigsqcup_{\pi \in P_i} \ell(\pi)$
κ	$\mathbb{T}$	state-grounded constraints K_i (policy / security / quota guards), same grounding
e	$\mathbb{T}$	effects $\bigoplus_{(x,v) \in E_i} \alpha(x := v)$
$\mathbf{i}$	$\{0, \frac{1}{2}, 1\}^{d_p}$	inputs: required = 1, optional = $\frac{1}{2}$
$\mathbf{o}$	$\{0, 1\}^{d_p}$	outputs, closed under domain hierarchy
τ	$\mathbb{N}^9$	one-hot capability type T_i
μ	$\mathbb{R}^{d_m}$	execution mechanism M_i : <i>signed feature hashing</i> of tokens key and key=value into $d_m = 64$ buckets
ρ	$\{0, 1\}^{d_r}$	resource requirements R_i
q^+	$\mathbb{R}_{\geq 0}^6$	additive attributes (t_{ms} , money, c_{res} , energy, h , λ) with hazard $h = -\ln(1 - \text{risk})$ and unreliability $\lambda = -\ln \text{Rel}$
q°	$\mathbb{R}^3$	availability (A , t_{start} , t_{end}) — flag and daily window for $A_i(t)$

Total dimension $d = 3d_a + 2d_p + 9 + d_m + d_r + 6 + 3$ (e.g. $d = 3 \cdot 26 + 2 \cdot 19 + 9 + 64 + 8 + 9 = 206$ for the checkout problem).

Requirement vector $r = p \sqcup \kappa$ and **guarantee vector** $g = e \oplus \sigma(r)$: what the capability needs before running, and what is certainly true after it has run (its effects, plus whatever it required and did not overwrite).

Design choices that answer the assignment's open questions

- *Type and mechanism.* T_i is a small one-hot block and M_i a separate hashed block. Both enter **only** the implementation similarity Sim_{impl} (weight 0.15 overall); they never enter compatibility, composition validity, or goal relevance. So an API and a database implementation of the same operation have identical functional blocks yet are *not* identical vectors.
- *Reliability.* Stored as $\lambda = -\ln \text{Rel}$ in the additive operational block. This makes reliability of a sequence a plain **sum** ($\prod \text{Rel}_i = e^{-\sum \lambda_i}$). It is **not** mixed into the semantic blocks.
- *Availability.* A min/max block q° ; composite availability = intersection of windows.
- *Cost, risk, resources.* Time, money, resource cost, energy are additive; risk is converted to a hazard (additive under independence); resource *requirements* are a set (union under composition).
- *Constraints.* State-grounded constraints form their own ternary block κ (same atom space as p), so *policy* can be weighted and inspected separately from *functional* preconditions. Constraints over several variables are declared as derived Boolean state variables (e.g. `Payment.within_limit`) and therefore can be *established* by another capability.

3. State-capability relationships

Transition operator (vector semantics of a capability).

$$\phi_S(\text{Apply}(S, E_i)) = \phi_S(S) \oplus e_i$$

Applicability. With $\text{need} = \{k : r_k \neq 0\}$, $N = |\text{need}|$:

$$\begin{aligned} \text{cov}(s, C) &= \frac{1}{N} |\{k \in \text{need} : s_k = r_k\}|, & \text{conf}(s, C) &= \frac{1}{N} |\{k \in \text{need} : s_k = -r_k\}|, \\ C \text{ applicable in } S &\iff \text{cov} = 1 \quad (\iff S \models P_i \wedge K_i). \end{aligned}$$

The scalar returned by `similarity(state, capability)` is $\text{cov} - \text{conf} \in [-1, 1]$.

4. Compatibility (directed) — precondition-effect and input-output

Compatibility answers “*can B run directly after A?*” and is deliberately **asymmetric**. Let $\text{ctx} = (s_0, \mathbf{m})$ optionally supply a known state s_0 (default 0 = unknown) and data already in the environment $\mathbf{m} \in \{0, 1\}^{d_p}$.

State part. $\hat{g} = g_A \sqcup s_0$ (what is guaranteed after A , plus what the context already knows). With requirement r_B and $N_B = \|r_B\|_1$:

$$\text{match} = \langle \hat{g}^+, r_B^+ \rangle + \langle \hat{g}^-, r_B^- \rangle, \quad \text{conflict} = \langle \hat{g}^+, r_B^- \rangle + \langle \hat{g}^-, r_B^+ \rangle,$$

$$\text{cov} = \text{match}/N_B, \quad \text{conf} = \text{conflict}/N_B$$

($x^+ = \max(x, 0)$, $x^- = \max(-x, 0)$ — the *dual-rail* split; all quantities are plain inner products, so an $N \times N$ compatibility matrix is two matrix products).

Supply (does A actually *contribute*?). Let $e_A^{new} = e_A$ restricted to entries where $e_A \neq \sigma(r_A)$ (genuine changes, not re-assertions of what A already required):

$$\text{supply} = \langle (e_A^{new})^+, r_B^+ \rangle + \langle (e_A^{new})^-, r_B^- \rangle.$$

Data part. Data available after A : $\mathbf{a} = \max(\mathbf{o}_A, \mathbb{1}[\mathbf{i}_A \geq 1], \mathbf{m})$; required inputs of B : $\mathbf{n}_B = \mathbb{1}[\mathbf{i}_B \geq 1]$:

$$\text{cov}_{io} = \langle \mathbf{a}, \mathbf{n}_B \rangle / \|\mathbf{n}_B\|_1, \quad \text{sup}_{io} = \langle \mathbf{o}_A, \mathbf{n}_B \rangle.$$

(an empty requirement gives coverage 1).

Label and score $\text{Comp}(A \rightarrow B \mid \text{ctx})$:

condition (checked in this order)	label	score
conflict > 0	CONFLICT	-conf
supply = 0 $\wedge$ sup _{io} = 0	INDEPENDENT (A contributes nothing to B)	0
cov = 1 $\wedge$ cov _{io} = 1	STRICT (A alone establishes everything B needs)	+1
otherwise	PARTIAL	min(cov, cov _{io})

With a known context state, “applicable after” is $\text{conf} = 0 \wedge \text{cov} = \text{cov}_{io} = 1$.

Why not just $\langle e_A, p_B \rangle$? (i) enum/integer *implied* literals need saturation; (ii) A’s own preconditions persist and may satisfy B; (iii) a net dot product hides one conflict behind one match; (iv) data dependencies live in a different space. The ablation in the report quantifies each.

5. Similarity (symmetric) — and why it is not composability

For two ternary/real blocks use the cosine with the convention $\cos(0, 0) = 1$, $\cos(0, x) = 0$. A block that is empty in *both* capabilities carries no information and is dropped from the weighted mean (weights renormalised over informative blocks).

$$\begin{aligned} \text{Sim}_{f_{unc}} &= \sum_{b \in \{p, \kappa, e, i, o\}} w_b \cos(b_X, b_Y), \quad (w_p, w_\kappa, w_e, w_i, w_o) = (.20, .10, .40, .10, .20) \\ \text{Sim}_{impl} &= .4 \cos(\tau) + .4 \cos(\mu) + .2 \cos(\rho) \\ \text{Sim}_{qos} &= 1 - \overline{|\tilde{q}_X - \tilde{q}_Y|}, \quad \tilde{q} = \left(\frac{\log(1+t)}{\log(1+t_{max})}, \frac{m}{m_{max}}, \frac{c}{c_{max}}, \frac{en}{en_{max}}, \text{risk}, 1 - \text{Rel} \right) \\ \text{Sim} &= 0.70 \text{Sim}_{f_{unc}} + 0.15 \text{Sim}_{impl} + 0.15 \text{Sim}_{qos}. \end{aligned}$$

($\tilde{q}$ is the *metric view* of the operational block: normalised per problem so that distances are meaningful; composition uses the raw additive block.)

Role of similarity. Sim answers “is Y a substitute / neighbour of X?” (alternative implementations, retrieval, failover). Comp answers “can Y follow X?”. Because Comp uses e_A against r_B (different blocks, opposite roles) and similar capabilities tend to share the *same* effect and *same* guard (so the second would be re-asserting / conflicting), high similarity does **not** imply composability: two payment implementations have $\text{Sim}_{f_{unc}} = 1$ and Comp = CONFLICT.

Single flat vector (for ANN indexing / downstream ML):

$$\Phi(C) = \bigoplus_b \sqrt{\omega_b} \hat{b}, \quad \hat{b} = b / \|b\|$$

with ω_b = overall weight $\times$ block weight; then $\cos(\Phi_X, \Phi_Y)$ is (up to the renormalisation of empty blocks) the weighted mean of block cosines. Empirically its correlation with Sim is 0.96–0.98 (report §9).

6. Composition

Let $X = C_1$ be executed first and $Y = C_2$ second. The composite $Y \circ X$ is computed **block by block** (so ϕ_C is closed: a composite is again a point of the same space and can be composed or compared again).

Validity. $g_X = e_X \oplus \sigma(r_X)$. The composition is *valid* iff $\nexists k : r_k^Y \neq 0 \wedge g_k^X = -r_k^Y$ (no conflict) and both parts are valid.

block	$Y \circ X$	meaning
p	$p_X \sqcup (p_Y \odot \mathbb{1}[e_X = 0])$	Y’s preconditions that X did <i>not</i> establish become requirements of the composite
κ	$\kappa_X \sqcup (\kappa_Y \odot \mathbb{1}[e_X = 0])$	same for constraints
e	$e_X \oplus e_Y$	last writer wins
i	$\max(i_X, i_Y \odot (1 - o_X))$	inputs not produced internally

block	$Y \circ X$	meaning
$\mathbf{o}$	$\max(\mathbf{o}_X, \mathbf{o}_Y)$	all outputs
τ, μ	$\tau_X + \tau_Y, \mu_X + \mu_Y$	bag of types / mechanisms
ρ	$\max(\rho_X, \rho_Y)$	union of resources
q^+	$q_X^+ + q_Y^+$	time, money, resource cost, energy, hazard, – In Rel add
q°	$(\min A, \max t_{start}, \min t_{end})$	available only when all are; window intersection

Because e_X sets *whole* variable groups, “touched by e_X ” is well defined and $\mathbb{1}[e_X = 0]$ is exact.

Chains. $\text{compose}([C_1, \dots, C_n]) = C_n \circ \dots \circ C_1$ by left fold.

Propositions (verified by the test-suite, §7 of the report):

1. *Closure.* $\phi_C(Y \circ X)$ lies in the same space as $\phi_C(C)$; the composite supports similarity, compat, compose, apply.
2. *Associativity.* $(Z \circ Y) \circ X = Z \circ (Y \circ X)$ block-wise, and validity agrees. (Sketch: the pre-block of both sides is $p_X \sqcup (p_Y \odot \neg t_X) \sqcup (p_Z \odot \neg t_X \odot \neg t_Y)$ with $t = \mathbb{1}[e \neq 0]$; effects are last-writer-wins; all other blocks are commutative-monoid operations.)
3. *Identity.* The all-zero capability (with window $[0, 24]$ and $A = 1$) is a two-sided identity.
4. *Soundness w.r.t. execution.* For every concrete state S : $\text{Applicable}(Y \circ X, S) \iff \text{Applicable}(X, S) \wedge \text{Applicable}(Y, \text{Apply}(S, E_X))$, and then $\phi_S(\text{Apply}(S, E_{Y \circ X})) = \phi_S(S) \oplus e_{Y \circ X}$ equals the state after running X then Y . (Sketch: preconditions of Y on variables written by X are decided by e_X ; all others must hold in S and are therefore preconditions of the composite.)
5. *QoS homomorphism.* $\text{Rel}(Y \circ X) = \text{Rel}_X \text{Rel}_Y$, $\text{risk} = 1 - (1 - r_X)(1 - r_Y)$ (independence), time/money/energy additive.

Composite vs. components. The composite is **not** the average or sum of its parts: $p_{Y \circ X}$ *loses* the literals X satisfies internally and keeps only the external requirements; $e_{Y \circ X}$ is dominated by later effects; the operational block is the monoid sum. A pooling baseline (mean/sum + clip) is evaluated in the report to show the loss.

7. Goal relevance

Direct alignment. With unmet-goal vector $u_k = \gamma_k$ if $\gamma_k \neq 0 \wedge s_k \neq \gamma_k$, else 0:

$$\text{align}(C \mid s, \gamma) = \frac{|\{k : e_k = u_k \neq 0\}| - |\{k : \gamma_k \neq 0, e_k = -\gamma_k\}|}{\|\gamma\|_1}.$$

align sees only capabilities whose effects hit a goal literal.

Regressed relevance (backward propagation in vector space — **a relevance score, not a plan search**). Maintain need-vectors $n^+, n^- \in \{0, 1\}^{d_a}$ and data needs $\mathbf{n}^d$, initialised with the unmet goal literals. Repeat for $t = 0, 1, \dots$: a capability *supplies* a need if $e_k = +1 \wedge n_k^+$ or $e_k = -1 \wedge n_k^-$ or $\mathbf{o} \wedge \mathbf{n}^d$; every newly supplying capability gets depth t , and its requirements that are not already true in the state (and its inputs not in the environment) join the needs. Stop at a fixed point ($\leq |C|$ rounds).

$$\text{rel}(C) = \begin{cases} \lambda^{\text{depth}(C)} & \text{if supplying and not harmful} \\ 0 & \text{otherwise} \end{cases} \quad (\lambda = 0.85),$$

where *harmful* means some effect contradicts a goal literal. State-awareness: requirements already true in s are not needed.

Cost: $O(|C| \cdot d_a)$ per round, $\leq |C|$ rounds, no sequences are enumerated.

8. Operational properties — embedded or separate?

attribute	where	composes by	affects compat?	affects similarity?
time, money, resource cost, energy	q^+ (additive)	sum	no	via Sim_{qos} (0.15)
risk	q^+ as hazard	sum of hazards	no	via Sim_{qos}
reliability	q^+ as $-\ln \text{Rel}$	sum	no	via Sim_{qos}
availability / window	q^+	min / max / min	no (filter)	no
resource requirements	ρ	union	no (feasibility filter)	via Sim_{impl}
constraints	κ (ternary)	merge-minus-touched	yes (guards block)	via Sim_{func} ($w = .1$)

Operational properties are used as *filters and rankings over functionally compatible/similar candidates*: $\text{feasible}(C, t, \mathcal{E}) = A \wedge t \in [t_s, t_e] \wedge \rho \leq \mathcal{E}$ (environment resources), and a utility $U(C; w) = w^\top \tilde{q}(C)$ (lower is better) with user-chosen weights w .

9. Complexity

- Storage per capability: d floats (≈ 0.8 KB at fp32 for the evaluated problems); sparse form $O(\text{literals})$.
- $\text{encode}(\text{capability})$: $O(|P| + |E| + |K| + |I| + |O|)$ ($\sim 15 \mu\text{s}$ measured).
- Pairwise compatibility $A \rightarrow B$: $O(d_a + d_p)$; all pairs: two dense matrix products $O(N^2 d_a)$ (or sparse).
- Composition of n capabilities: $O(n d)$.
- Goal relevance: $O(|C|^2 d_a)$ worst case.

10. How the eight required properties are met

#	property	mechanism	evidence
1	identity	block vectors differ if pre/eff/ports/mechanism differ	distinct vectors; alt. implementations differ in μ, τ, ρ only (Exp. 3)
2	state awareness	ϕ_S lives in the same atom space; $\phi_S \oplus e$; coverage	0 mismatches over >9 000 trials (Exp. 2b)
3	pre/eff compatibility	match, conflict, supply	100 % agreement with exhaustive oracle (Exp. 1)
4	input/output compatibility	port vectors with hierarchy closure	ablation “no ports” (Exp. 1)
5	similarity $\neq$ composability	separate symmetric Sim and directed Comp	AUC of similarity-based predictors 0.52–0.68 vs 0.87–0.96 for directed Comp (Exp. 1)
6	composition	block-wise monoid homomorphism	closure, associativity, soundness (Exp. 2)
7	goal relevance	ϕ_G , align, regressed relevance	mean F1 0.98–1.00 for regressed relevance vs 0.33–0.91 for direct alignment (Exp. 4)
8	operational properties	separate monoid block, filters	Exp. 5